

使用 Spring Resource 抽象層存取 Cloud Storage 檔案

來源：<https://codelabs.developers.google.com/codelabs/spring-cloud-gcp-gcs?hl=zh-tw>

步驟數：7

瞭解如何使用 Spring Resource 抽象存取 Cloud Storage 中的檔案。

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [在 Cloud Storage 中建立檔案](#)
4. [初始化 Spring Boot 應用程式](#)
5. [讀取 Cloud Storage 中的檔案](#)
6. [寫入 Cloud Storage 中的檔案](#)
7. [恭喜！](#)

步驟 1 · 約 1 分鐘

1. 總覽

Spring Framework 提供 `ResourceLoader` 抽象化功能，可輕鬆讀取及寫入各種來源的檔案，例如檔案系統、類別路徑或網路。您只需要使用眾所周知的通訊協定前置字串，指定資源的 URI。舉例來說，如要存取本機檔案系統中的檔案，您會指定類似 `file:/data/config.yaml` 的 URI。

您將編寫 Spring Boot 應用程式，使用 Spring Resource 抽象層和 `gs:` 通訊協定前置字串存取儲存在 Cloud Storage 的檔案。

您將使用 Cloud Shell 和 Cloud SDK `gcloud` 指令列工具完成這項操作。

課程內容

- 如何使用 Cloud Storage Spring Boot 啟動器
- 如何使用 Spring 存取 Cloud Storage 中的檔案
- 如何使用 Spring 的 `Resource` 和 `WritableResource` 抽象概念

軟硬體需求

- 具備 Google Cloud 專案
 - 瀏覽器，例如 [Google Chrome](#)
 - 熟悉標準 Linux 文字編輯器，例如 Vim、Emacs 和 GNU Nano
-

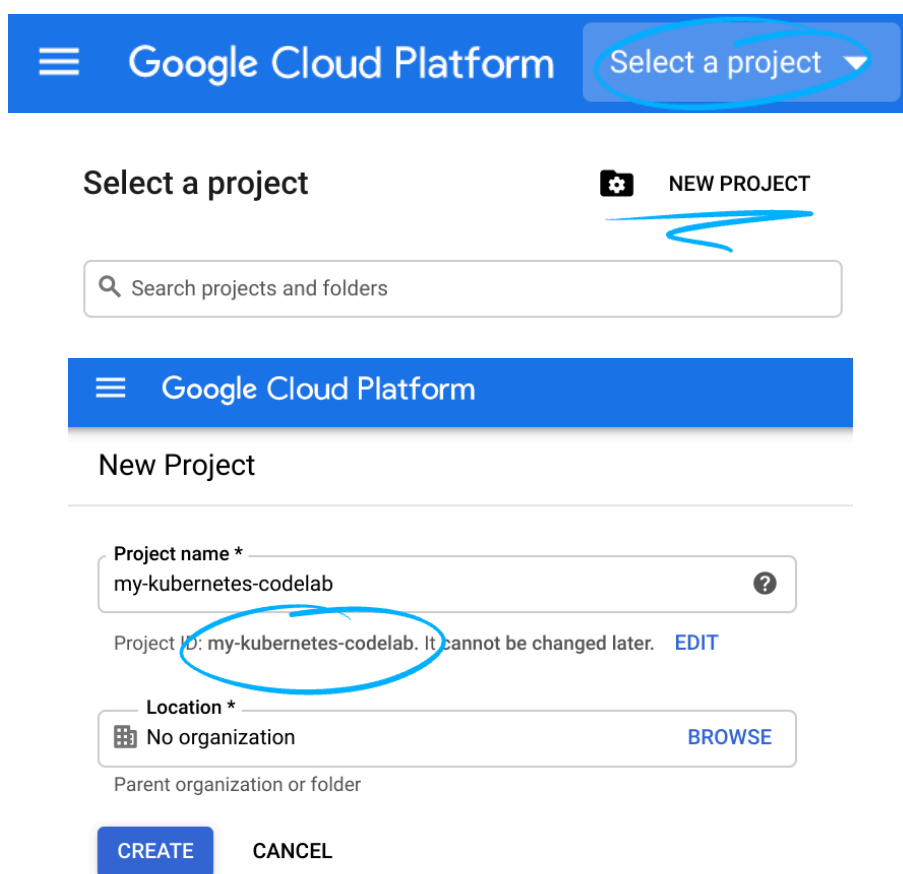
步驟 2 · 約 1 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。(如果沒有 Gmail 或 G Suite 帳戶，請先[建立帳戶](#))。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：console.cloud.google.com。



The screenshot shows the Google Cloud Platform interface for creating a new project. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown. Below this, the 'Select a project' section includes a search bar and a 'NEW PROJECT' button. The 'New Project' form has three main fields: 'Project name' (containing 'my-kubernetes-codelab'), 'Project ID' (containing 'my-kubernetes-codelab' and circled in blue), and 'Location' (set to 'No organization'). There are also 'CREATE' and 'CANCEL' buttons at the bottom.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 G Suite 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

Cloud Shell

您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。

啟用 Cloud Shell

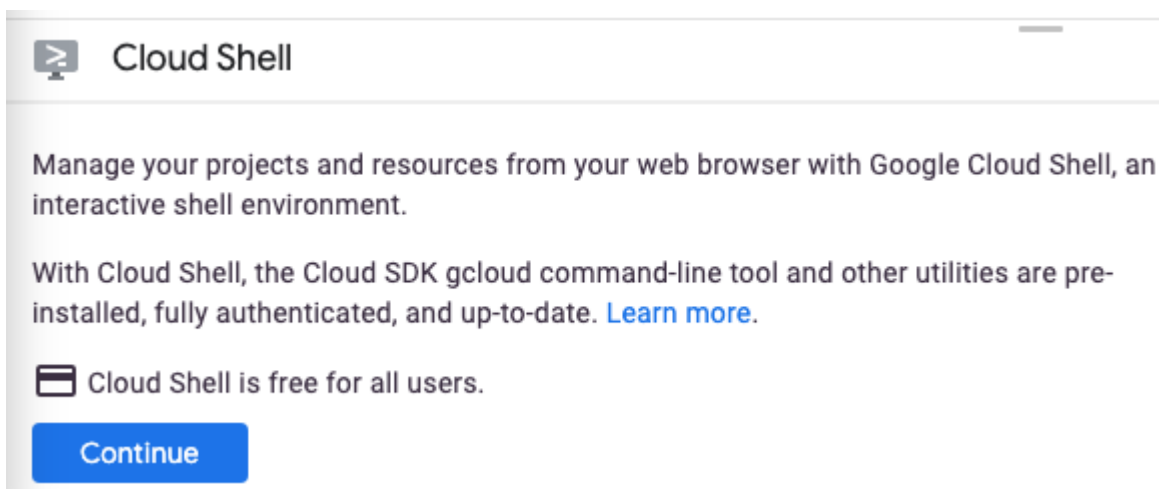
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (需捲動位置)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



```
Welcome to Cloud Shell! For help, visit https://cloud.google.com/cloud-shell/help.  
testuser003@codelab-test003:~$ gcloud auth list
```

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

步驟 3 · 約 2 分鐘

3. 在 Cloud Storage 中建立檔案

啟動 Cloud Shell 後，您就可以開始建立檔案，並將檔案轉移至 Cloud Storage。

建立名為 `my-file.txt` 的檔案：

```
$ echo "Hello World from GCS" > my-file.txt
```

接著在 Cloud Storage 中建立新的專屬 bucket，並使用 `gsutil` 將檔案轉移至該 bucket。

```
$ BUCKET=spring-bucket-$USER  
$ gsutil makebucket gs://$BUCKET  
$ gsutil copy my-file.txt gs://$BUCKET
```

前往 [Cloud Storage 中的儲存空間瀏覽器](#)，確認 bucket 和檔案都在其中。

步驟 4 · 約 2 分鐘

4. 初始化 Spring Boot 應用程式

使用指令列，透過 Spring Initializr 產生新的 Spring Boot 應用程式，開始編寫應用程式：

```
$ curl https://start.spring.io/starter.tgz \
  -d type=maven-project \
  -d dependencies=web,cloud-gcp-storage -d baseDir=spring-gcs | tar -xzvf -
```

請注意，Initializr 會自動將 `spring-boot-starter-web` 和 `spring-cloud-gcp-starter-storage` 新增至範本應用程式 `pom.xml` 中的依附元件。

變更為範本應用程式的目錄：

```
$ cd spring-gcs
```

確認 `JAVA_HOME` 已設為正確的 JDK 版本：

```
$ export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64/
```

使用 Maven 建構並執行應用程式。

```
$ ./mvnw spring-boot:run
```

應用程式會開始監聽通訊埠 8080。開啟新的 Cloud Shell 分頁，然後執行 `curl` 存取應用程式。

```
$ curl localhost:8080
```

由於應用程式尚未執行任何實用動作，因此您應該會收到 404 回應。

返回應用程式執行的前一個 Cloud Shell 分頁，然後使用 `Control+C` (Macintosh 上的 `Command+C`) 終止應用程式。

步驟 5 · 約 4 分鐘

5. 讀取 Cloud Storage 中的檔案

修改 Spring Boot 應用程式，存取先前儲存在 Cloud Storage 中的 `my-file.txt` 檔案。您的目標是透過 HTTP 傳回檔案內容。

在下列操作說明中，您將使用 Vim 編輯檔案，但也可以使用 Emacs、GNU Nano 或 Cloud Shell 內建的程式碼編輯器：



```
$ cd ~/spring-gcs
```

在應用程式中新增 REST 控制器 `GcsController`。

```
$ vi src/main/java/com/example/demo/GcsController.java
```

貼上下列程式碼，別忘了使用先前建立的 bucket 修正資源 URI。您可以執行 `echo $BUCKET` 指令來檢查 bucket。

src/main/java/com/example/demo/GcsController.java

```
package com.example.demo;

import java.io.IOException;
import java.nio.charset.Charset;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.io.Resource;
import org.springframework.util.StreamUtils;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GcsController {

    @Value("gs://REPLACE_WITH_YOUR_BUCKET/my-file.txt")
    private Resource gcsFile;

    @GetMapping("/")
    public String readGcsFile() throws IOException {
        return StreamUtils.copyToString(
            gcsFile.getInputStream(),
            Charset.defaultCharset());
    }
}
```

使用 Maven 建構及執行應用程式：

```
$ ./mvnw spring-boot:run
```

應用程式會開始監聽通訊埠 8080。開啟新的 Cloud Shell 分頁，然後執行 `curl` 存取應用程式。

```
$ curl localhost:8080
```

現在您應該會看到應用程式傳回的檔案內容。前往應用程式執行的前一個 Cloud Shell 分頁，然後使用 `Control+C` (Macintosh 上的 `Command+C`) 終止應用程式。

步驟 6 · 約 3 分鐘

6. 寫入 Cloud Storage 中的檔案

您讀取 Cloud Storage 中的檔案內容，並透過 Spring REST 控制器公開該內容。現在，將新檔案內容發布至相同的 HTTP 端點，即可變更檔案內容。

您需要在 `GcsController` 中新增另一個方法，用來回應 HTTP POST 並將資料寫入 Cloud Storage 中的檔案。這次請將 Spring `Resource` 轉換為 `WritableResource`。

使用您需要的其他匯入項目更新 `GcsController`。

src/main/java/com/example/demo/GcsController.java

```
import java.io.OutputStream;
import org.springframework.core.io.WritableResource;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.PostMapping;
```

將新的端點方法新增至控制器。

src/main/java/com/example/demo/GcsController.java

```
@RestController
public class GcsController {

    @PostMapping("/")
    String writeGcs(@RequestBody String data) throws IOException {
        try (OutputStream os = ((WritableResource) gcsFile).getOutputStream()) {
            os.write(data.getBytes());
        }
        return "file was updated\n";
    }
    ...
}
```

使用 Maven 建構及執行應用程式：

```
$ ./mvnw spring-boot:run
```

應用程式會開始監聽通訊埠 8080。開啟新的 Cloud Shell 分頁，然後執行 `curl`，將訊息發布至應用程式。

```
$ curl -d 'new message' -H 'Content-Type: text/plain' localhost:8080
```

系統應會顯示確認訊息，說明檔案內容已更新。不過，請執行 `GET` 進行驗證。

```
$ curl localhost:8080
```

您應該會看到應用程式傳回的檔案內容已更新。返回應用程式執行的前一個 Cloud Shell 分頁，然後使用 `Control+C` (Macintosh 上的 `Command+C`) 終止應用程式。

步驟 7 · 約 0 分鐘

7. 恭喜！

您已學會使用 Spring Resource 抽象層，輕鬆存取 Cloud Storage 中的檔案。您編寫的 Spring Boot 網頁應用程式可以讀取及寫入 Cloud Storage 中的檔案。您也瞭解了 Cloud Storage 的 Spring Boot 啟動器，可啟用這項功能。

瞭解詳情

- [Cloud Storage](#)
- [在 Google Cloud 中使用 Spring Framework](#)
- [Google Cloud 上的 Spring GitHub 存放區](#)
- [在 Google Cloud 中使用 Java](#)

授權

這項內容採用的授權為 [創用 CC 姓名標示 2.0 通用授權](#)。
